

Release Candidate 2

Briefing Pack v1.0

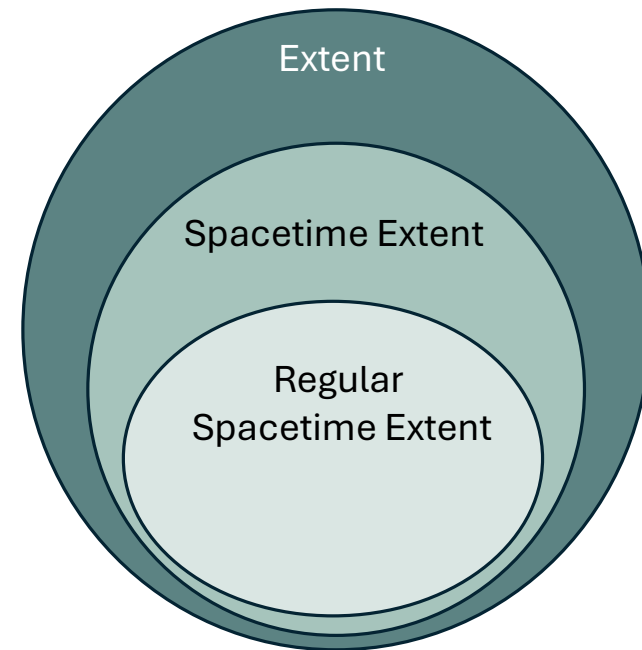
(July 2026)



<https://github.com/IES-Org/ies-top>

Changes in RC2

1. Introduction of *Extent* as the root of the “individuals” hierarchy. In RC1, the hierarchy of individuals started with *SpatiotemporalExtent* (which has also been renamed – see (3)). RC2 introduces *Extent*, allowing for individuals that are not necessarily spatiotemporal. This change makes explicit IES-Top's commitment to Extensionalism, upon which Four-Dimensionalism is subsequently built. It also moves IES-Top closer to David Lewis' principle of *plenitude*, under which “the worlds are many and varied” and may differ fundamentally from our own. In Lewis' terms, this includes worlds in which “totally different laws govern the doings of alien particles with alien properties”¹. Consequently, *SpatiotemporalExtent* is now a subclass of *Extent*.
2. Introduced explicit *WorldboundExtent* and *TransworldExtent* classes. In RC1, these distinctions were represented through class definitions; they are now explicit classes in their own right.
3. The class *SpatiotemporalExtent* has been renamed to *SpacetimeExtent*. This makes clearer, that at this level, we are talking about “chunks” of spacetime (in a spacetime world). Moreover, the terminology is now aligned with the foundational spacetime literature originating with Minkowski and Einstein, while also providing a shorter and cleaner term in the RDF implementation.
4. The class, *RegularSpacetimeExtent* has been introduced as a subclass of *SpacetimeExtent*. This change provides the foundation for updates to IES-Core in relation to spatial objects, relative spaces and coordinate systems. Note: in practice, virtually all usage of IES relates to regular spacetime extents.
5. Related to (1), RC2 introduces a new class, *World*, as the superclass of the existing *Universe* class. This allows for worlds irrespective of if they are spacetime worlds or not.
6. A refinement to the pattern for continuous and intermittent states, giving it a standard mereotopological foundation based on *self-connection* and *self-disconnection* — which is itself built on the newly introduced *connected* and *disconnected* relations.



1. D. K. Lewis, *On the plurality of worlds*. Malden, Mass: Blackwell Publishers, 1986.

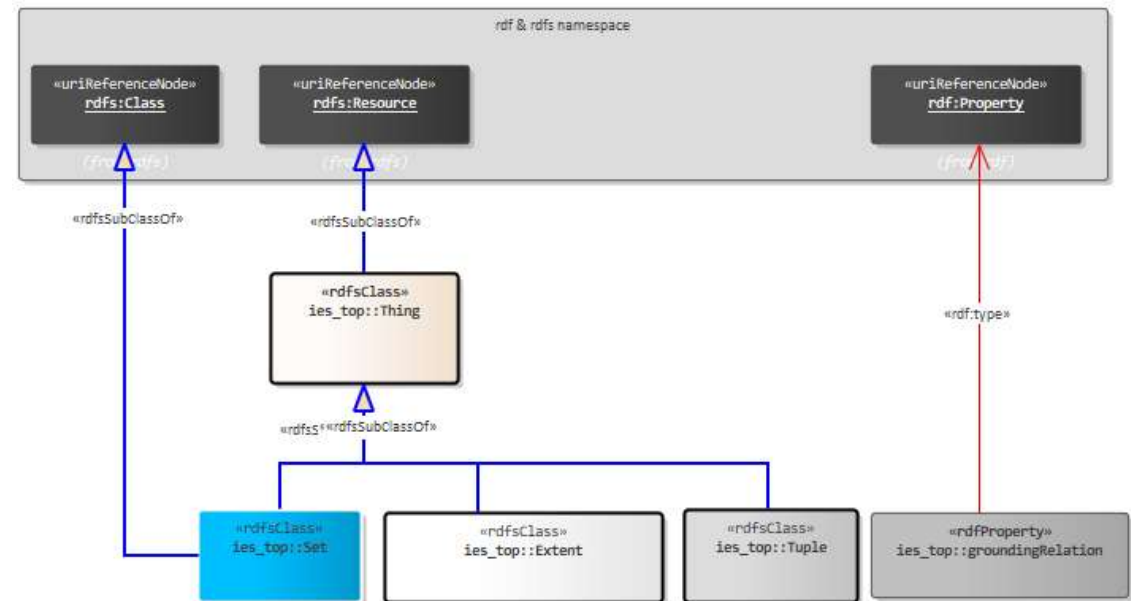
Design decisions

IES-Top is based on the topics of Extensional Four-Dimensionalism, Pluralities and Constructionalism. Translating these into a physical implementation (like RDF) inevitably requires making choices—whether due to limitations of the chosen implementation technology or for user experience reasons. Throughout this document, we call out design decisions made in this regard in purple call-out boxes.

Introducing IES-Top

The top of Top

- **Thing** – Is either an extent, set, tuple or grounding relation.
- **Extent** - A Thing that is a part of the pluriverse – and so has an extent.
- **Set** – A Thing that is an unordered collection of other Things, sometimes referred to as a class or kind.
- **Tuple** - A Thing that is an (ordered) sequence of Things. Each position in the sequence is pointed to by a tuplePlace.
- **groundingRelation** – A thing which is one of the four basic relationships between two Things.



DESIGN DECISIONS:

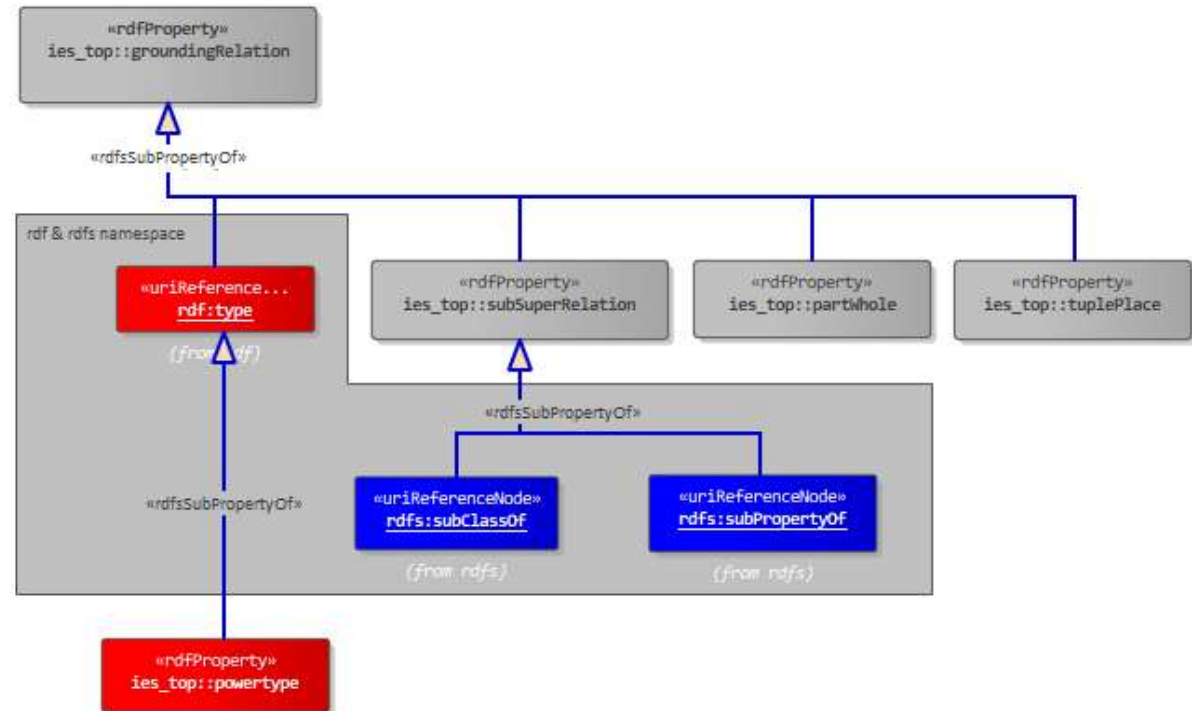
1. The equivalent top level of the BORO™ Foundational Ontology and the Core Constructional Ontology, introduce higher-order relations that go beyond those available in RDF/S. At this level we are not talking about sub and super sets/classes but sub and super pluralities. However, in the implementation we have avoided adding such relations and instead stuck with using subClassOf and subPropertyOf which are ubiquitous among the RDF community and its tools.
2. In IES4, Class was used instead of Set. The use of the word Class encouraged users to think in an Object Orientated Programming way which led to confusion. In IES-Next we have elected to use the word Set to encourage users to think in a set-theoretic way.
3. The equivalent item to groundingRelation in the BORO™ is a sub-plurality just like a Tuple and Set. Because all grounding relations in the ToLO have one source and one target, we have realized groundingRelation in RDF, for simplicity, as an rdf:Property.

Grounding Relations

Grounding relations are derived from the four basic constructors outlined in the Core Constructional Ontology (CCO)². They are:

1. type (Element-Set)
2. subSuperRelation (Subset-Superset)
3. partWhole (Part-Whole)
4. tuplePlace (Tuple Place)

Note: regarding the above, the first name is that used in IES-Top, the name in brackets is used in CCO. In addition: colloquially in IES we sometimes refer to *type* as *Member-Set*.



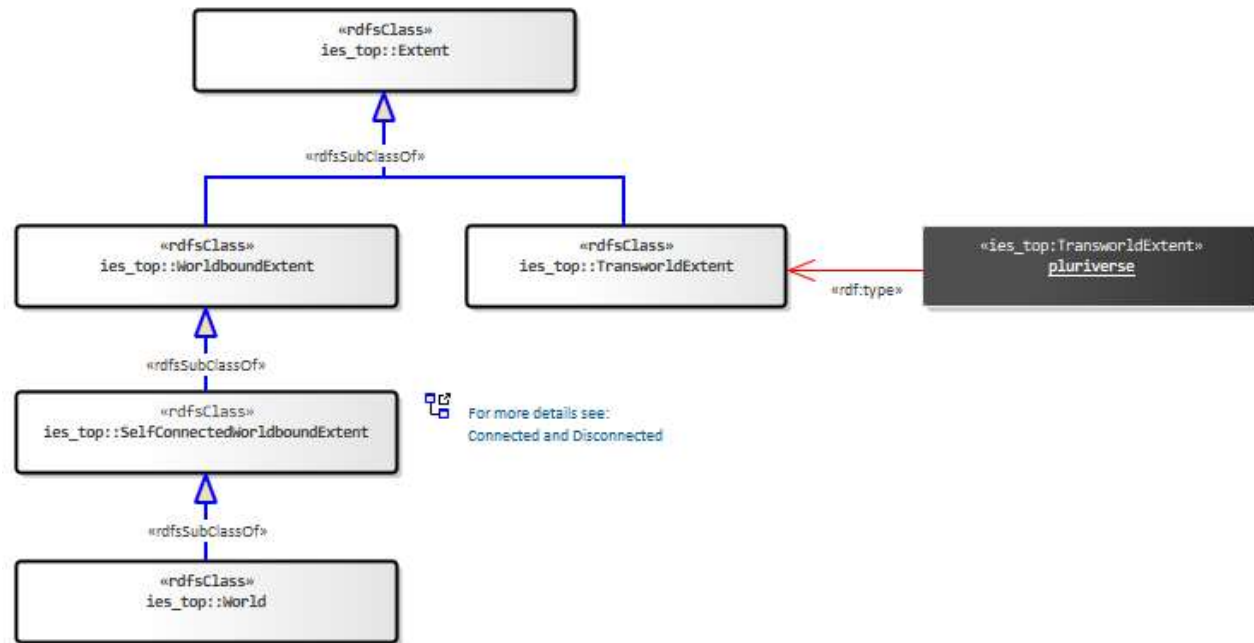
Note: As mentioned in *Design Decision 3* on the previous slide, the concept of pluralities is found at a higher-order than the concepts in RDF/S. As a result, we have had to define RDF/S resources in the context of our top-level. Here you will see that `rdf:type`, `rdfs:subClassOf` and `rdfs:subPropertyOf` are themselves `ies_top:groundingRelations`.

2. Florio, S., & Linnebo, Ø. (2021). *The many and the one: A philosophical study of plural logic* (First edition). Oxford University Press.

The Pluriverse

In IES, we commit to the *Pluriverse*, which provides the foundation for talk about possibilities (and Intensional Properties). We use an approach based on David Lewis' interpretation of *Possible Worlds*¹ (which we refer to as *Worlds*) to make clear distinctions between extents that can be *trans-world* (part of more than one world) and the ones we do most of our work with, those that are *world-bound* (part of one and only one world). This commitment also accommodates worlds that are radically dissimilar to our own, including worlds governed by different laws of nature, topological structures, and other fundamental features.

- **WorldboundExtent** – an extent that is bound to one world i.e. part of one and only one world.
- **TransworldExtent** – an extent that has parts in more than one world. Note: all transworld extents are self-disconnected*.
- **World** – a maximal self-connected* worldbound extent that includes everything in a world, irrespective of any indexing, such as whether it is present now, in the past, or in the future.
- **pluriverse** - An instance of TransworldExtent which is the maximal sum of all extents and so is the sum of all worlds (and their parts). Put another way, this has everything in every world as a part. This includes worlds made up of spacetime as well as worlds with non-spatiotemporal structures.



1. D. K. Lewis, *On the plurality of worlds*. Malden, Mass: Blackwell Publishers, 1986.

* see slide on [Connected and Disconnected](#)

Identity

In extensional ontologies, like IES, the identity of an individual is determined by its extension or extent:

If two individuals have the same parts, they are the same individual.

This provides the (mereological) foundation for the spatiotemporal criterion of identity, first introduced into IES at Version 4.

If two individuals occupy the same spacetime, they are the same individual.

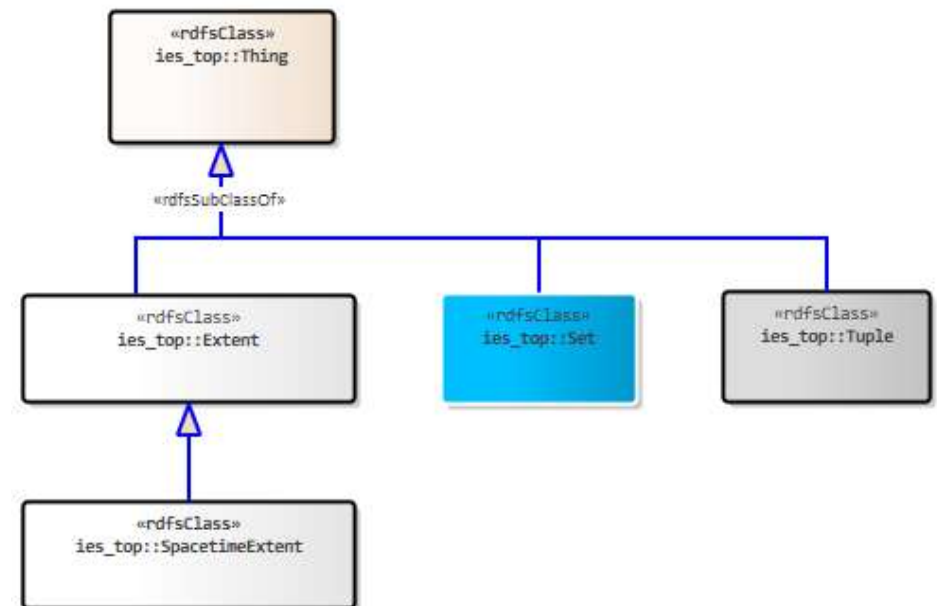
IES-Top makes these identity criteria explicit within its hierarchy through the introduction of *Extent*, representing any part of a pluriverse, and its subclass *SpacetimeExtent*, representing any part of a spacetime world*. The extensional approach to identity also applies to the other two forms of Thing: Sets and Tuples.

The criterion of identity for Set is:

If two sets have the same members, they are the same set.

And for Tuple:

If two tuples have the same sequence of members, they are the same tuple.



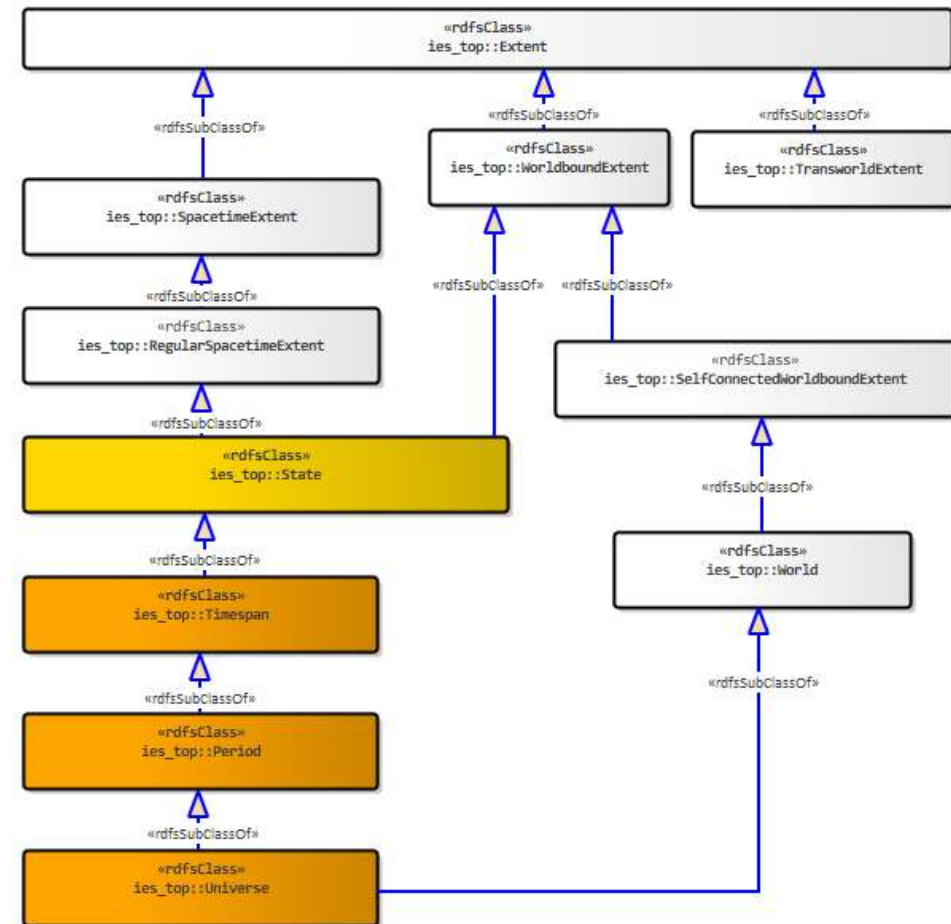
* for more details, see slide on [Spacetime](#)

Spacetime

SpacetimeExtent denotes any *Extent* that is part of a spacetime world, which is a world with a *before-after* relation. This is intentionally general and does not presuppose any particular geometrical, topological, or differential structure of the spacetime world.

Nevertheless, most work in physics, including General Relativity, represents spacetime as a smooth four-dimensional manifold. Since the principal applications of IES are situated within such spacetimes, IES-Top explicitly distinguishes between *SpacetimeExtent* in general and *RegularSpacetimeExtent*, whose underlying spacetime satisfies these regularity conditions.

- **SpacetimeExtent** – a spatiotemporal extent aka. a four-dimensional extent.
- **RegularSpacetimeExtent** – a spacetime extent which is part of a world which is a smooth four-dimensional manifold (a universe).
- **Universe** – a regular spacetime world i.e., one with a smooth four-dimensional manifold.
- **State** – a regular spacetime extent that is universe-bound i.e. part of one and only one universe.
- **Timespan** – a state (i.e. universe-bound regular spacetime extent) that is a temporal part of a universe. Note: a universe is an improper temporal part of itself – and so a maximal timespan.
- **Period** – a self-connected and therefore uninterrupted timespan.

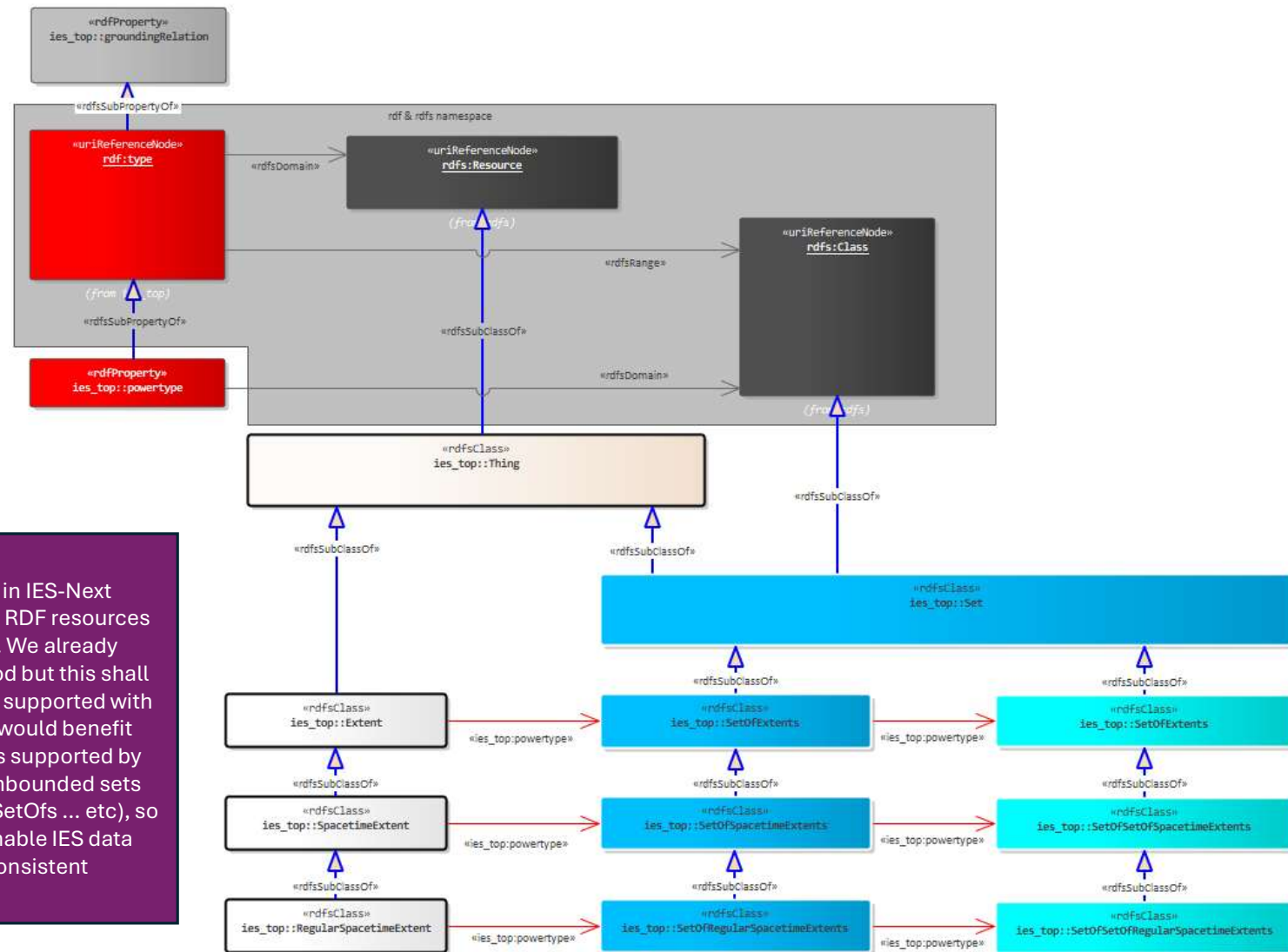


DESIGN DECISIONS:

4. In naming RDF resources, IES-Top strikes a balance between two competing aims: names that are familiar and easy for users, and names that are explicit about what they denote from a 4D and Possible Worlds perspective. *SpacetimeExtent* and *State* being two examples of this. IES4's *Element* has been renamed *SpacetimeExtent* – which is explicitly what it always was. Here we chose the more explicit name. By contrast, the extents users deal with 99% of the time are regular spacetime extents bound to a single universe. Rather than naming this ubiquitous class with an unwieldy name like *UniverseBoundRegularSpacetimeExtent*, we carry over IES4's *State*, now qualified precisely as a *RegularSpacetimeExtent* bound to a single universe. Here we chose familiarity.

Instances and types

To support the placement of elements into sets, we adopt the ubiquitously used *rdf:type* relation. This follows the same approach as IES4. *powertype* also carries over from IES4.



DESIGN DECISIONS:

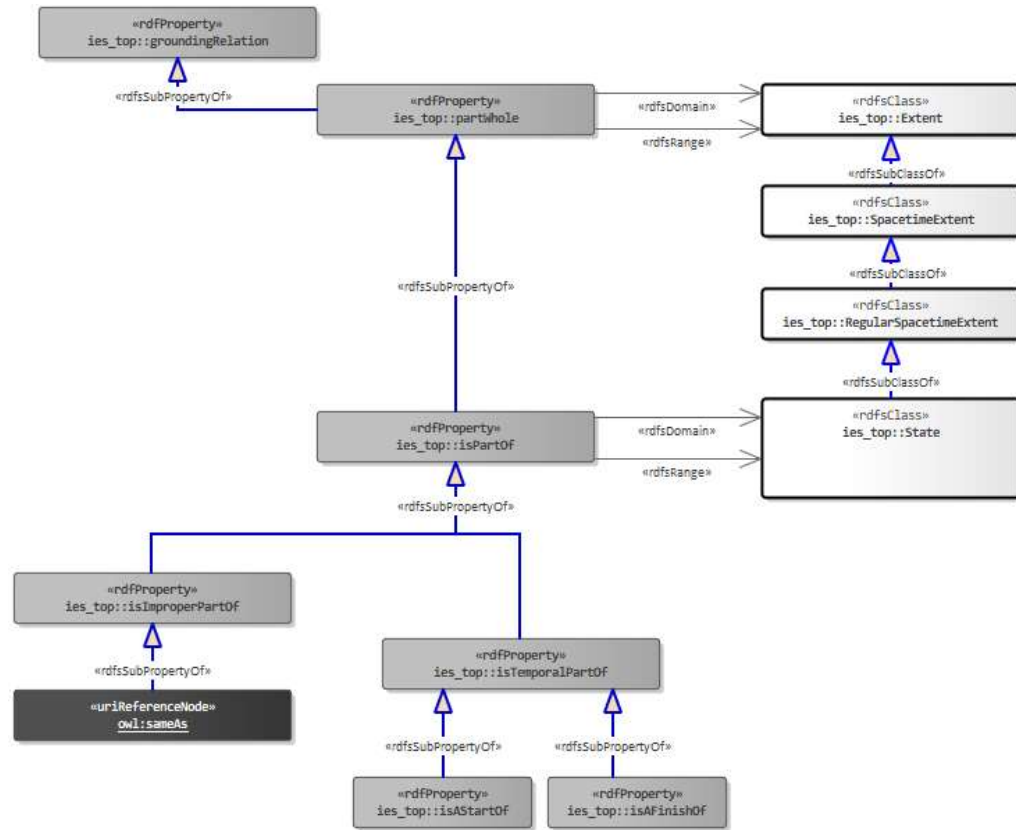
5. A new URI minting regime will be included in IES-Next which will support the consistent creation of RDF resources many users typically create within their data. We already have such a pattern for IES4's ParticularPeriod but this shall be extended to other areas of the model and supported with SHACL validation artefacts. One area which would benefit from this is the unbounded, infinite set levels supported by IES. It is not feasible to issue a model with unbounded sets (e.g. SetOfs, SetOfSetOfs, SetOfSetOfSetOfSetOfs ... etc), so a URI minting pattern shall be specified to enable IES data creators to create the levels they need in a consistent fashion.

Parts and wholes

IES-Top provides mereological relations at two levels of generality. The base relation, *partWhole*, places one extent as part of another and holds between extents of any kind - whether they are bound to a single world or span across two worlds. The relations most users will utilize - **isPartOf** and its sub-relations - are narrower: they hold between regular spacetime extents, both bound to a single universe (universe-mates) - that is, between states.

- **partWhole** – a grounding relation placing one extent as part of another (the whole).
- **isPartOf** – a partWhole relation between two states, where both states are bound to the same universe (universe-mates).
- **isTemporalPartOf** – an isPartOf that asserts the spatial extent of the (whole) state is co-extensive with the spatial extent of the (part) state for a particular period of time.
- **isAStartOf** – an isTemporalPartOf that places a state as one (but not always the only) temporal part at the beginning of another.
- **isAFinishOf** – an isTemporalPartOf that places a state as one (but not always the only) temporal part at the conclusion of another.
- **isImproperPartOf** – an isPartOf between two states that are the same i.e., the part is identical to the whole.

Note: this final mereological relation provides the foundations for OWL's *sameAs* relation between individuals.



DESIGN DECISIONS:

- Another example of *Design Decision 4* is the use of the terms *partWhole* and *isPartOf*. *isPartOf* is carried over from IES4 for familiarity reasons to be the *partWhole* relation users will commonly use.
- isStateOf* is replaced with the more explicit *isTemporalPartOf* term for IES-Top.
- To simplify querying, IES-Next reduces the number of specialised sub-properties of *partWhole*. For example, commonly used properties such as *inLocation*, *inPeriod*, and *isParticipantIn* are replaced by *isPartOf*, while *isParticipationOf* is replaced by *isTemporalPartOf*. This removes the need for queries to enumerate multiple specialised *partWhole* relations simply to retrieve all parts of an extent.
- Because *BoundingStates* in IES4 are time intervals, the set of boundings can collapse into many things. Consider a Person - Birth, Conception, Childhood are obvious Start boundings of persons. But the same would go for Adulthood and even Person itself - they are starting boundings of something. The same goes for finish/end boundings. Persons can die during childhood or even during birth. To avoid this issue of collapse we have opted to not carry over the *partWhole* relations of *isStartOf* and *isEndOf* from IES4. These relations could be interpreted as stating that this is one and only one start/end bounding. Instead, we provide the clearer relations of *isAStartOf* and *isAFinishOf*. This makes explicit that any instances of these relations are just one of many starts or ends.

Tuples and couples (1 of 2)

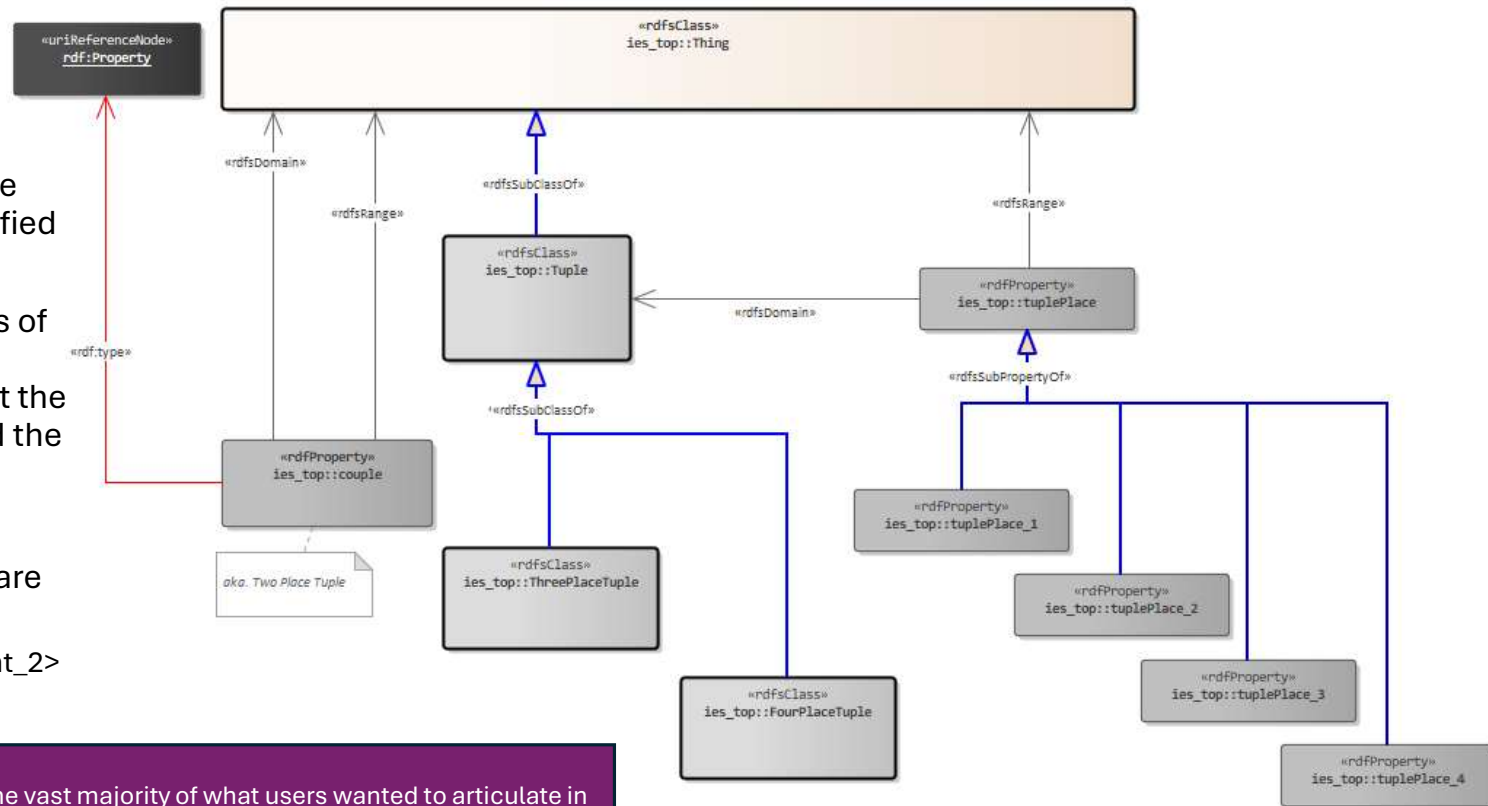
A tuple is a sequence of two or more things. Each part of a tuple is identified by a *tuplePlace*.

E.g. for the tuples that are members of *Father-Son Tuples*, you recover the father-son relations by knowing that the first tuple place is for the father and the second for the son:

<father_1, son_1>

Another example is the tuples that are members of the *Between Tuples*:

<endpoint_1, midpoint_x, endpoint_2>



DESIGN DECISIONS:

9. For IES4, we avoided higher arity Tuples as the vast majority of what users wanted to articulate in data are two-placed tuples aka. Couples. Couples were realized using simple RDF properties and this will be the same in IES-Next. However, in IES-Next we want to have a solid and complete top-level and that means having tuples that are beyond 2 places. As a result, we will support 2-placed tuples using the user-friendly RDF properties and beyond-2 placed tuples using the RDF N-ary approach which we explain in [appendix 1](#).

Tuples and couples (2 of 2)

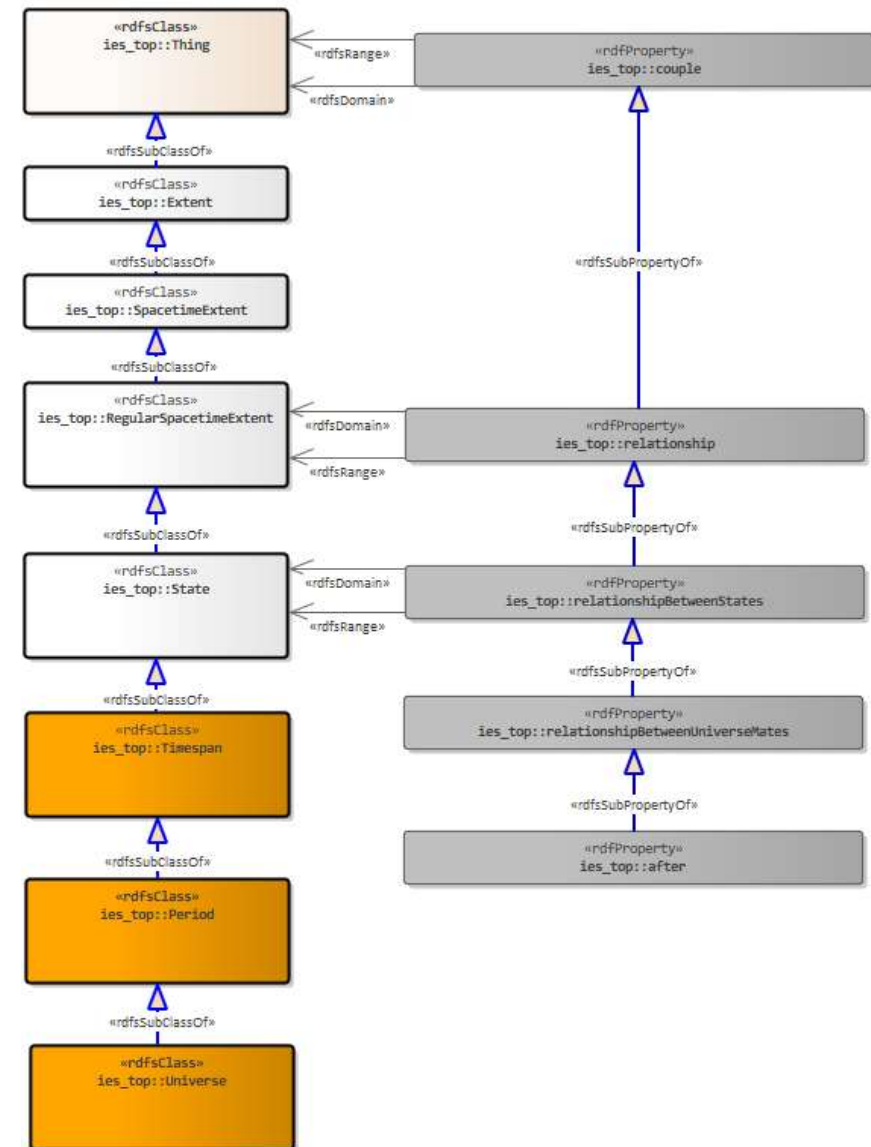
As with the mereological relations, IES-Top provides couple relations at several levels of generality, drawing the same world-bound distinctions: **relationship** is a couple between any two regular spacetime extents - whether or not they are universe-bound. **relationshipBetweenStates** narrows this to universe-bound extents (states), and **relationshipBetweenUniverseMates** narrows it further to states belonging to the same universe (universe-mates).

- **couple** – a two placed tuple. Realized in RDF as an rdf:property.
- **relationship** – a couple between any two regular spacetime extents.
- **relationshipBetweenStates** – a relationship between any two states, where a state is a universe-bound regular spacetime extent.
- **relationshipBetweenUniverseMates** – a relationship where those states are part of the same universe as one another.
- **after** – a precedence relation between universe-mates where one lies ahead of the other in time: every part of the later one has some part of the earlier one behind it, every part of the earlier one has some part of the later one ahead of it, and nothing runs back the other way.

Couples are utilized for relationships between extents and sets, or between two sets.

DESIGN DECISIONS:

4. Another example of *Design Decision 4* is the naming of relationshipBetweenStates and relationshipBetweenUniverseMates. A balance again has been struck between being clear with what they refer to and user-friendly naming. Admittedly, the naming of these relationships has been somewhat harder than the previous examples.



Connected & Disconnected

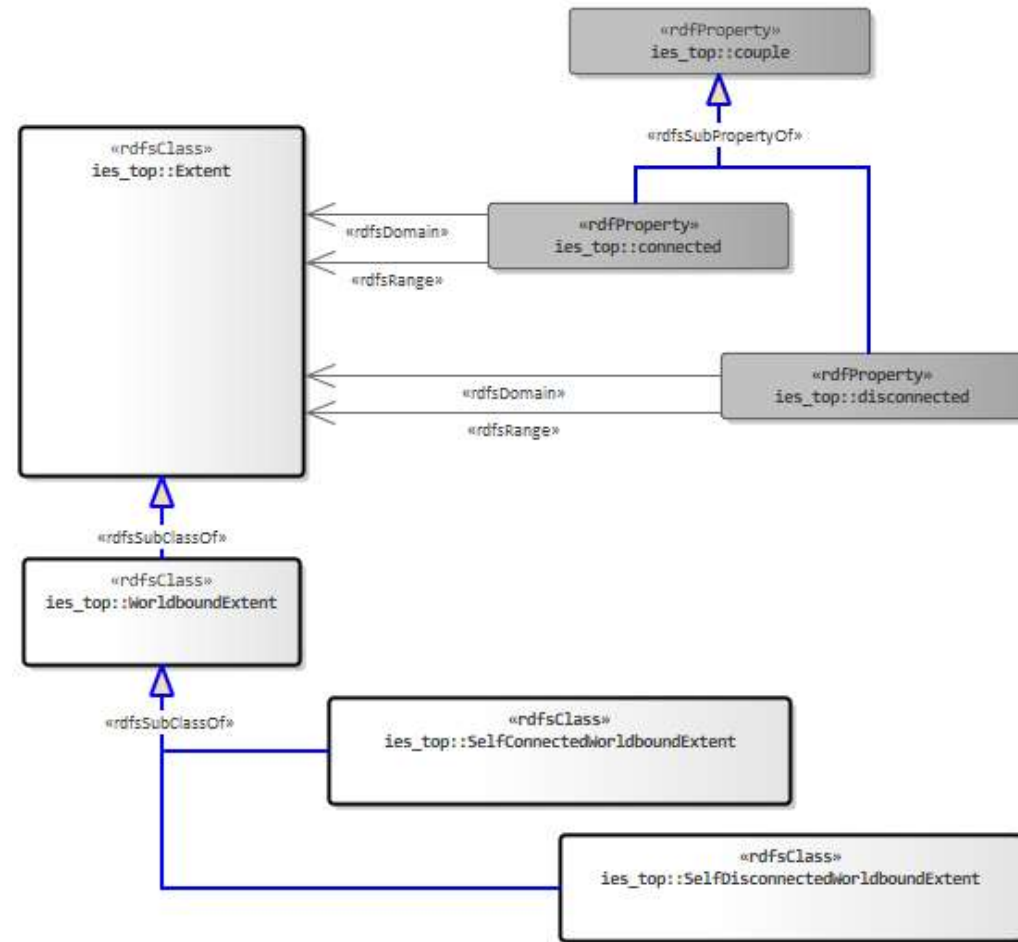
IES4 did have means of talking about individuals that are temporally uninterrupted (no gaps in their temporal extent) and those that are gappy. But that distinction lacked a foundation. We resolve this in IES-Top by providing a standard mereotopology answer to the question of gaps: **connected** and **disconnected**.

Two extents are **connected** if they are in contact - meeting with no gap between them - and **disconnected** if a gap separates them.

Connection is symmetric and is not the same as overlap*: extents that overlap are connected, but so are ones that merely abut, like two bricks side by side. Because connection tells us when two things touch, it also lets us ask whether a 4D extent is all one piece or is instead made up of separate chunks.

In fact, a 4D extent can be "in pieces" in two different ways: it can be spread out across space at any one time e.g., a fleet of ships, scattered but sailing together; or it can be broken across time e.g., a trumpet that is assembled, taken apart, and reassembled. Standard mereotopology uses the term *self-connected*, if an individual can't be split into two separate parts, and *self-disconnected*, if it can.

- **connected** - a couple relation between two extents that are in contact, meeting with no gap between them. This couple relation is symmetric.
- **disconnected** - a couple relation between two extents that are not in contact, i.e., they are separated by a gap. This couple relation is symmetric.
- **SelfConnectedWorldboundExtent** - a worldbound extent where there is always a path that connects any two parts. Put another way: whichever way it is cut, the two parts always touch. They will never be separated.
- **SelfDisconnectedWorldboundExtent** - a worldbound extent that can be cut into two parts that do not touch i.e., they are separated by a gap. It is made up of separate chunks.

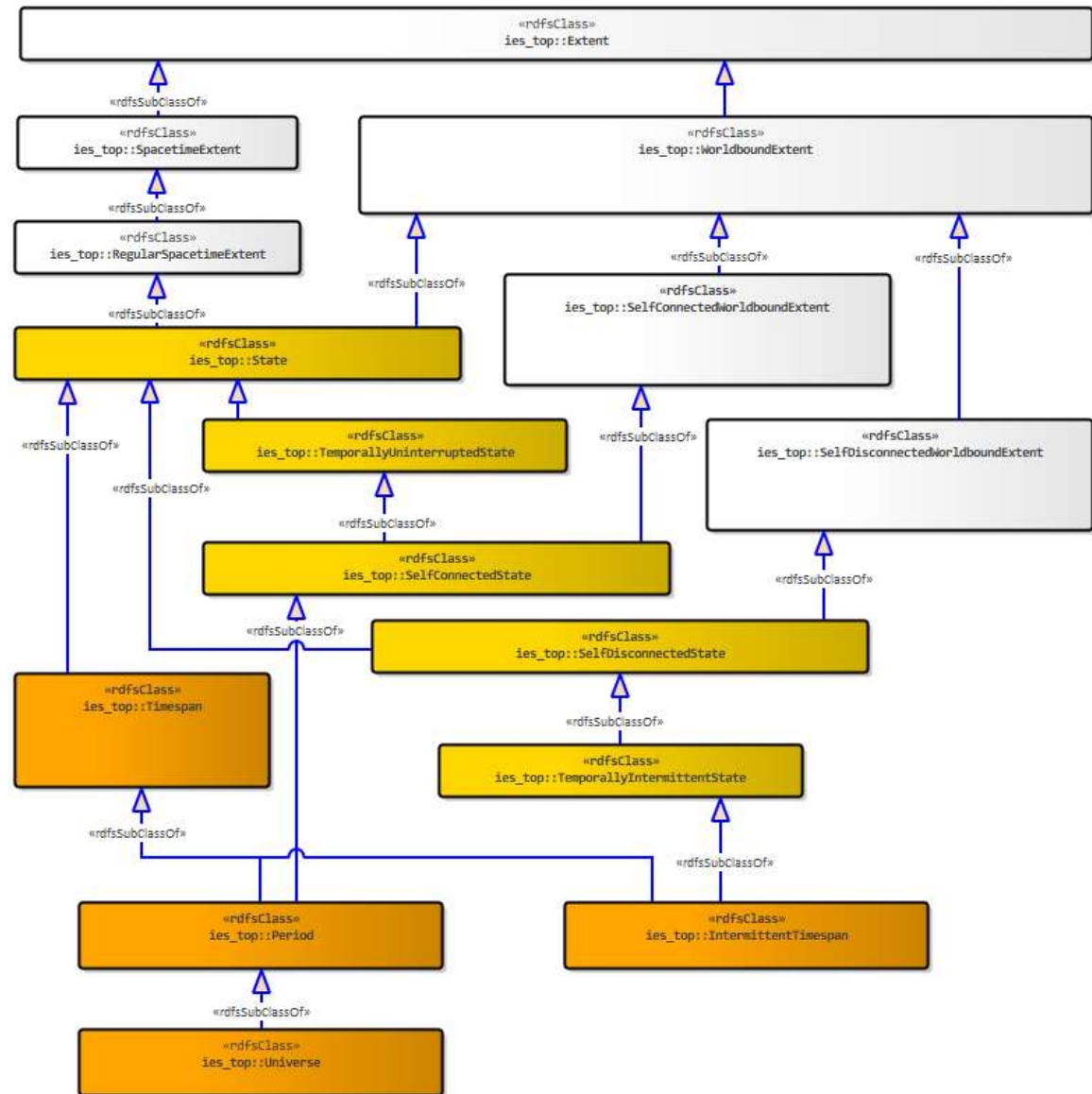


* see slide on [Overlap and Disjoint](#)

Uninterrupted & Intermittent

Now that we have a foundation for talking about gaps and self-connectedness, we can provide an explanation for states that are temporally uninterrupted (no gaps in their temporal extent) and those that are gappy.

- **TemporallyUninterruptedState** – a state with no gap in time. Whichever way it is cut into an earlier part and a later part, the two always meet i.e., there is no temporal moment between them where it does not exist.
- **SelfConnectedState** – a temporally uninterrupted state that, as well as having no gap in time, has no gap in space. Whichever way it is cut, the two always touch; it is one connected whole.
- **SelfDisconnectedState** – a state that is a single whole that can be cut into two parts that do not touch; separated by a gap. It is made up of separate chunks rather than being one continuous extent.
- **TemporallyIntermittentState** – a self-disconnected state with a gap, or gaps, in time. It can be cut into an earlier part and a later part that do not meet.
- **Timespan** – a state (i.e. universe-bound) that is a temporal part of a universe. Note: a universe is an improper temporal part of itself – and so a maximal period.
- **Period** – a self-connected and therefore uninterrupted timespan.
- **IntermittentTimespan** – an interrupted timespan which is also a fusion of timespans.

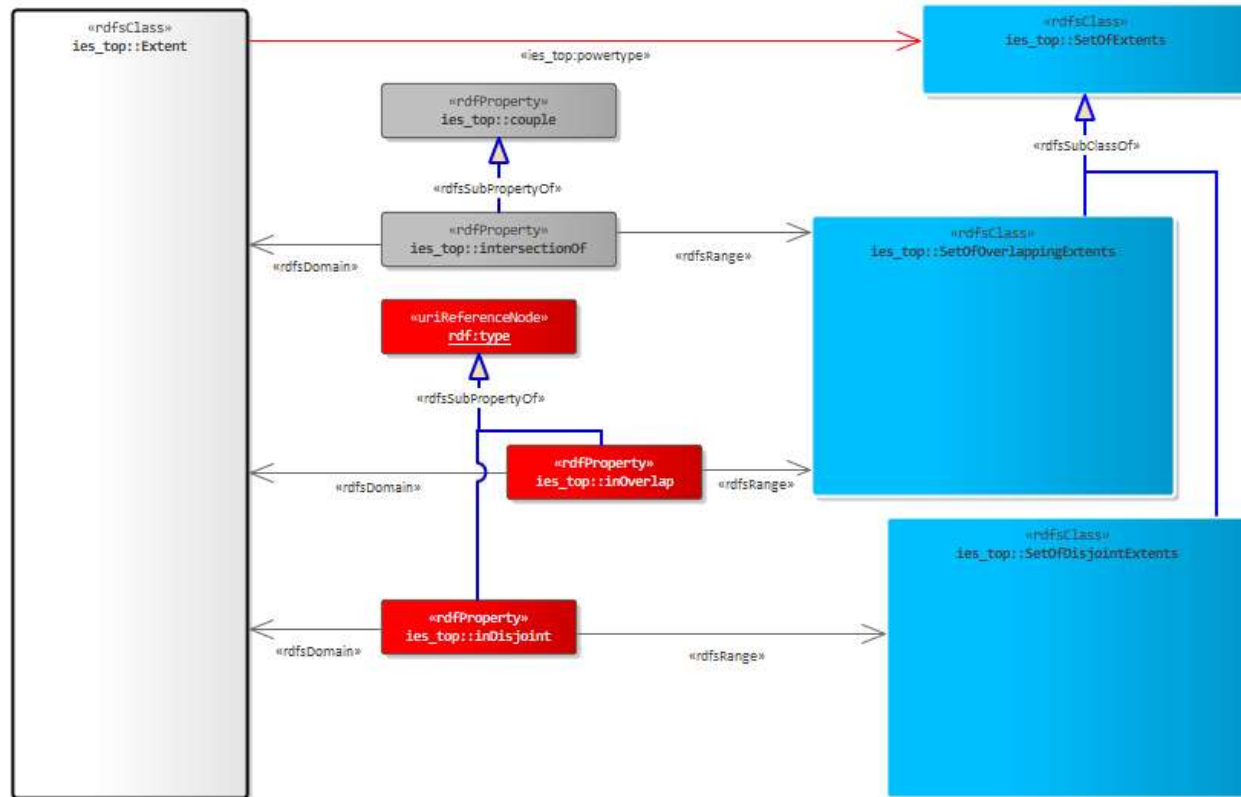


Overlap and Disjoint

There are times when two or more extents have shared parts and expressing that shared part (the intersection) is useful e.g., coverage area of two mobile phone cellular masts.

In other cases, it is equally important to call out two or more extents that have no shared parts i.e. are disjoint. For example, disjoint paths taken by two aircraft. Note that disjoint extents can be connected*.

* see slide on [Connected and Disconnected](#)

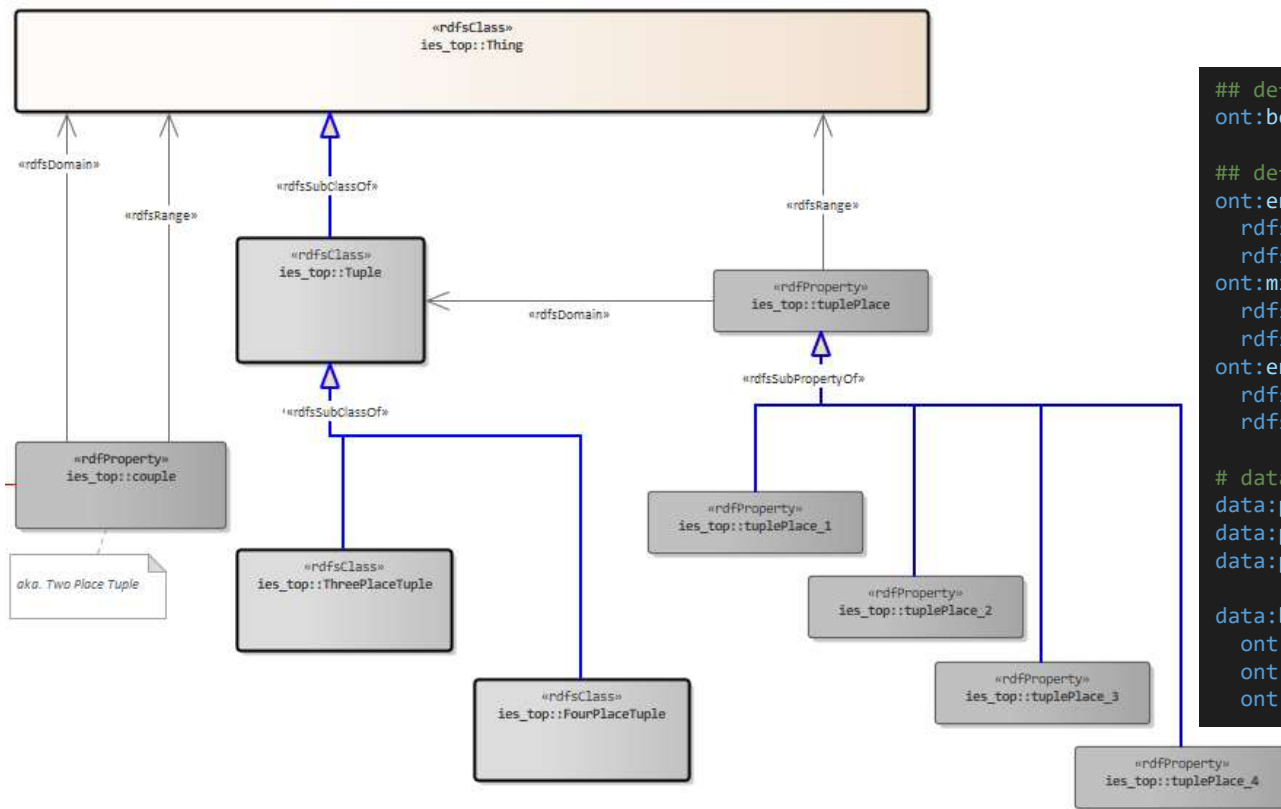


DESIGN DECISIONS:

10. This decision complements Decision 7. Whereas IES-Next reduces specialised relationships when they unnecessarily complicate querying, it introduces specialised relationships where they surface information that is normally considered separately from an extent's *conventional* type. This is the case here, with `inOverlap` and `inDisjoint` (subProperties of `rdf:type`). This allows queries about an extent's commonly understood type (e.g. `Person` or `Vehicle`) to remain separate from queries about its membership of overlap or disjoint sets.

Appendix 1: Using tuples

Here we demonstrate how to specify a three-placed tuple (*Between*) in IES (using the N-ary RDF approach). We also demonstrate the use of this newly specified tuple to describe the relationship of a person between 2 other persons in a queue/line.



```

## define a new three place tuple
ont:between rdfs:subClassOf ies_top:ThreePlaceTuple .

## define the place types and their domain and range
ont:endPoint1 rdfs:subPropertyOf ies_top:tuplePlace_1 ;
rdfs:domain ont:between ;
rdfs:range ies_core:Person .
ont:midpoint rdfs:subPropertyOf ies_top:tuplePlace_2 ;
rdfs:domain ont:between ;
rdfs:range ies_core:Person .
ont:endPoint2 rdfs:subPropertyOf ies_top:tuplePlace_3 ;
rdfs:domain ont:between ;
rdfs:range ies_core:Person .

# data that utilised newly defined between tuple type:
data:person_1 a ies_core:Person .
data:person_2 a ies_core:Person .
data:person_3 a ies_core:Person .

data:between_2 a ont:between ;
ont:endPoint1 data:person_1 ;
ont:midpoint data:person_2 ;
ont:endPoint2 data:person_3 .

```